

Document number: P3415R0
Date: 2024-10-10
Audience: LEWG
Reply-to: Andrzej Krzemiński <akrzemi1 at gmail dot com>

Range interface in `std::optional` breaks code!

[P1255R12] provided a motivation for treating optional objects as ranges of zero or one elements in certain contexts, and proposed a solution for *opting into* such treatment via a dedicated adapter. [P3168R2] went further and proposed *forcing* the treatment of optional objects as ranges, even in places where this is not desired, without offering a strong enough motivation. In this paper we argue that this "improvement" is harmful, and that it will break existing libraries and user code. We propose to revert changes in [P3168R2] and to reconsider [P1255R12], which is not intrusive.

1. The model behind `std::optional`

I am one of the authors of [N3793], author of the [reference implementation](#) of `std::optional` and a maintainer of [Boost.Optional](#) library. This paper is based on experience with a real library servicing many customers, private and commercial. After the years of experience with `Boost.Optional` and `std::optional`, I conclude that its design is broken. The main reason, as Tony Van Eerd observed years ago, is that it does not have a one clear answer to the question what it is. Sometimes it is a container of `T`, sometimes it is just `T` with an additional value `nullopt`. These two models can coexist to some degree, but at some point they clash and `std::optional` will surprise the user. For instance, one can expect of the container, as a safety feature, that a container cannot be compared to its element:

```
return std::vector{1} == 1; // will not compile
```

Even though we could provide quite intuitive semantics to such comparison (size of 1 and the contained element equals the value), we do not do it, in order to protect the user from accidental unintended usages of such comparison. However, `std::optional` — considered a container by some — does offer such mixed comparison, sometimes causing harmful effects:

```
optional<double> Flight_plan::weight();  
    // `nullopt` when we cannot compute weight  
  
bool is_aircraft_too_heavy(Flight_plan const& p)  
{  
    double max_weight = p.aircraft().max_weight();  
    return p.weight() > max_weight; // compiles!  
}                                     // returns `false` on `nullopt`
```

We cannot fix this, as this would be a breaking change, but what we can do is to avoid making this rift between different models bigger. We should not pretend that `std::optional` is a container and add a range interface to it, while at the same time pursuing optional references ([P2988R7]). No container supports references. If a component is just a bag of features without a clear model, people cannot predict how it will interact with other features.

[P3168R2] gives an impression that the trade-off is between adding a new adapter type and not adding a new type. But a more accurate description would be that the trade-off is between adding a new adapter type and compromising the integrity of an existing type, and breaking the currently working code, as demonstrated in the next section.

2. Breaking libraries and user code

The present C++ Standard Library provides concepts like `std::ranges::range`, therewith encouraging and endorsing the SFINAE-style detection of the range interface and making some compile-time decisions. It is reasonable to assume that people already check if user types model `std::ranges::range` and select different overloads. Any such code may automatically break when C++ suddenly makes `std::optional` a range.

Interestingly, one such undesired effect was discovered in the Standard Library itself, where `optional<int>` would become formattable as a range. So the range formatting protocol has been explicitly disabled for `optional<int>`. The Standard Library can afford to disable it for its own usages at the same time as introducing the change. But other libraries cannot.

As an evidence of this breakage, consider library `Boost.JSON`. It offers a way to convert a user type to a JSON "value". This is documented [here](#). A JSON value is one of a couple of categories (string, number, integer, boolean, array, object), and each converted user-defined type needs to be assigned one of these categories. This assignment is based on the overload resolution. Being a range renders category "array" and is prioritized over being optional-like, which renders category "object".

```
std::optional<int> o1 = 16;  
json::value v1 = json::value_from(o1);           // (1) store optional as JSON value (integer)  
std::string s = json::serialize(v1);              // (2) serialize to text: `16`  
json::value v2 = json::parse(s);                 // (3) parse text to JSON value (integer)  
auto o2 = json::value_to<std::optional<int>>(v2); // (4) convert JSON value integer to optional
```

This illustrates the use case where we want to serialize our data into JSON text representation, and then recreate our object on the other side from the JSON text representation. Function `json::value_from` is overloaded. First, an overload is considered that tests the type predicate `is_sequence_like<T>`, which fails, and then the type predicate `is_optional_like<T>`, which succeeds. This means that `std::optional<int>` is considered a JSON integer, and printed as string ``16``.

If `std::optional` becomes a range one day, the categorization above will change. Type trait `is_sequence_like<T>` will return `true`, and `std::optional<int>` will be treated as a JSON array, producing text output ``[16]``. This is a breaking change of the worst kind: the program still compiles but has different semantics. The different JSON output may no longer be schema-compliant, and break its specification. It will be discovered late in the game, because compilation succeeds, initial tests may pass. The tickets will be filed against Boost.JSON library rather than the C++ Standard.

There is a second breaking change here. If this new value is read back into `std::optional<int>`, function `json::value_to` using the same trait `is_sequence_like<T>`, will select the *sequence protocol* for parsing the array and call:

```
o2.insert(value, o2.end());
```

And this will now fail to compile. This will break the existing programs that use Boost.JSON today.

Here is a full example in Compiler Explorer: <https://godbolt.org/z/5YW36bcex>.

The fix to this breakage, for Boost.JSON, is to special-case the type trait `is_sequence_like<T>` so that it detects all range-like types *except for* `std::optional<T>`. This repeats the same pattern as the fix for `std::format`: after `std::optional<T>` is made a range, libraries have to explicitly request that it should not be treated as a range. It is somewhat ironic that while the goal of [P3168R2] is to make `std::optional<T>` model concepts like `std::ranges::range`, the libraries negatively impacted by it, will have to now devise a new concept, "any range but optional objects".

3. Profit and loss balance

How does this cost weigh against the benefits? The primary motivation in [P1255R12] is to have additional building blocks for ranges. There is a secondary motivation: the usage of `for`-loops instead of `if`-statements, but we consider it a stretch. Besides, pattern matching ([P2688R2]) will serve this purpose better. Thus, usages other than in ranges will at best be unaffected and at worst be damaged.

Users who compose range pipelines are already used to drawing components from the `ranges` namespace, and it is clear that the single purpose of these views is to be a building block in range pipelines. Adding building blocks in `ranges` seems natural and intuitive, even if the implementation is similar to other components.

Should we care about breaking user code?

[SD-8] argues that if we avoided *any* potential breaking changes too literally, we wouldn't be able to add anything to the Standard Library, because any change can be detected via SFINAE tricks and, in the future, reflection. It lists specifically that the Standard Library reserves the right to add members to the reserved namespaces and to classes in these namespaces.

However, we claim that the situation in question is special and requires an individual approach. This is because the Standard library offers the concept `std::ranges::range`. With this, the Standard Library recognizes that members, or hidden friends, `begin()` and `end()` have special meaning for the Standard Library. It recognizes that the user code will be testing for the presence of these members. `std::ranges::range` encourages testing for these functions. The change of the Standard Library component to start modeling a Standard Library concept is a **big** change.

4. The proposal

We propose to revert the changes applied by [P3168R2].

Whether [P1255R12] should be added instead, is beyond the scope of this proposal.

5. Proposed wording

The proposed wording is relative to [N4988].

Remove declarations from [optional.syn] as follows.

```
namespace std {
    // [optional.optional], class template optional
    template<class T>
        class optional;                                // partially freestanding

template<class T>
constexpr bool ranges::enable_view<optional<T>> = true;
template<class T>
constexpr auto format_kind<optional<T>> = range_format::disabled;

    template<class T>
```

```

    concept is-derived-from-optional = requires(const T& t) { // exposition only
        []<class U>(const optional<U>&){ }(t);
    };
    // ...
}

```

Remove declarations from [\[optional.optional.general\]](#) as follows.

```

namespace std {
    template<class T>
    class optional {
    public:
        using value_type          = T;
        using iterator          = implementation-defined; // see [optional.iterators]
        using const_iterator    = implementation-defined; // see [optional.iterators]

        // ...

        // [optional.swap], swap
        constexpr void swap(optional&) noexcept(see below);

        // [optional.iterators], iterator support
        constexpr iterator begin() noexcept;
        constexpr const_iterator begin() const noexcept;
        constexpr iterator end() noexcept;
        constexpr const_iterator end() const noexcept;

        // [optional.observe], observers
        constexpr const T* operator->() const noexcept;
        // ...
    };
}

```

Remove clause [\[optional.iterators\]](#).

Remove a macro definition from [\[version.syn\]](#), header `<version>` synopsis, as follows.

```

#define __cpp_lib_optional          202110L // also in <optional>
#define __cpp_lib_optional_range_support 202406L // freestanding, also in <optional>
#define __cpp_lib_out_ptr          202106L // also in <memory>

```

6. Acknowledgments

Barry Revzin, Tim Song and Jens Maurer offered useful suggestions that increased the quality of the proposal.

7. References

- [N3793] — Fernando Cacciola, Andrzej Krzemiński, "A proposal to add a utility class to represent optional objects (Revision 5)" (<https://www.open-std.org/jtc1/sc22/wg21/docs/papers/2013/n3793.html>).
- [P1255R12] — Steve Downey, "A view of 0 or 1 elements: `views::maybe`" (<https://www.open-std.org/jtc1/sc22/wg21/docs/papers/2024/p1255r12.pdf>).
- [P2688R2] — Michael Park, "Pattern Matching: `match` Expression" (<https://www.open-std.org/jtc1/sc22/wg21/docs/papers/2024/p2688r2.html>).
- [P2988R7] — Steve Downey, Peter Sommerlad, "`std::optional<T>`", (<https://www.open-std.org/jtc1/sc22/wg21/docs/papers/2024/p2988r7.pdf>).
- [P3168R2] — Marco Foco, Darius Neațu, Barry Revzin, David Sankel, "Give `std::optional` Range Support", (<https://www.open-std.org/jtc1/sc22/wg21/docs/papers/2024/p3168r2.html>).
- [SD-8] — Bryce Lebach, "Standard Library Compatibility", (<https://isocpp.org/std/standing-documents/sd-8-standard-library-compatibility>).
- [N4988] — Thomas Köppe, "Working Draft Programming Languages — C++" (<https://www.open-std.org/jtc1/sc22/wg21/docs/papers/2024/n4988.pdf>).